

Laboratório de Concorrência · Aula 10

Eduardo Barella · RA 24088000-2 · Turma A

Gerado em 09/10/2026, 20:50:36 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação é quando uma tarefa depende do resultado da outra, a tarefa A espera a tarefa B terminar porque só depois disso consegue continuar. Competição as tarefas são independentes, mas disputam o mesmo recurso, que só pode ser usado por uma de cada vez.

Exemplos: cooperação: o padeiro coloca os pães na prateleira e o balconista vende. O balconista não pode vender o pão se a prateleira estiver vazia, então ele espera o padeiro repor. E o padeiro não pode colocar mais pães se a prateleira estiver cheia. Um depende do outro para continuar.

competição: banheiro com uma chave só. Duas pessoas que não têm nada a ver uma com a outra precisam usar o mesmo banheiro. Quem chega primeiro entra e tranca a porta, e a outra espera do lado de fora até ele sair. Elas não dependem do resultado uma da outra só não podem usar o banheiro ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero que imprima 2.000.000, porque cada thread soma 1.000.000. Mas como as duas mexem no mesmo contador ao mesmo tempo e contador++ não é uma operação só, acho que pode sair um valor menor e diferente a cada execução.

1ª execução (n = 1.000.000)

1289381

2ª execução (n = 1.000.000)

1421462

3ª execução (n = 1.000.000)

1980019

Saída do programa

```
n = 1.000.000
esperado: 2000000
obtido: 1289381 / 1421462 / 1980019 (3 execuções)
```

```
n = 1000
esperado: 2000
obtido: 2000 / 2000 (2 execuções)
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram. O esperado era 2.000.000 e eu obtive 1.289.381, 1.421.462 e 1.980.019. Todos ficaram abaixo do esperado e nenhum se repetiu. Os valores mudam porque quem decide quando cada thread executa, e por quanto

tempo, é o escalonador, não o programa. A cada execução as duas threads se intercalam de um jeito diferente, então a quantidade de somas perdidas também muda. É a condição de corrida, o resultado depende de qual thread chega primeiro.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Supondo que o contador vale 100:

1. A thread 1 lê o contador (100) e guarda no seu registrador.
2. Antes de a thread 1 gravar, o escalonador passa a vez para a thread 2, que também lê 100.
3. A thread 1 soma 1 no registrador dela: 101.
4. A thread 2 soma 1 no registrador dela: 101.
5. A thread 1 grava 101 no contador.
6. A thread 2 grava 101 no contador, por cima do valor da thread 1.

As duas threads somaram 1, mas o contador só subiu de 100 para 101. Uma das somas se perdeu, porque a thread 2 trabalhou com um valor antigo. Isso acontece milhares de vezes durante a execução, e por isso o total fica bem abaixo de 2.000.000.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$ o resultado deu 2000 nas duas execuções, certinho. Com tão poucas repetições, cada thread termina o laço muito rápido, às vezes antes de a outra começar, então elas quase nunca leem e gravam ao mesmo tempo. Isso torna o erro perigoso porque ele é intermitente. Nos testes, com poucos dados e poucos usuários, o programa parece funcionar e passa. Em produção, com muitos acessos ao mesmo tempo, as threads se sobrepõem e os valores começam a sair errados de vez em quando, por exemplo um saldo ou um estoque. Como o erro nem sempre aparece, ele é difícil de reproduzir e de encontrar.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
BBBAAA → 7
ABABAB → 6
BABABA → 4
AABBAB → 6
AABBBA → 4

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBB
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis (as maneiras de intercalar os 3 passos de A com os 3 passos de B) e aparecem 4 resultados diferentes: 4, 6, 7 e 8. Só 1 das 20 ordens dá o resultado considerado correto, 8 (AAABBB). O 7 também sai de uma ordem só (BBBAAA), e o 4 e o 6 saem de 9 ordens cada. Ou seja, em 18 das 20 ordens uma das operações se perde.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Escolhi a ordem AABBBBA:

1. A lê TOTAL: registrador de A = 3.
2. A calcula: $3 + 1 = 4$ (ainda não gravou).
3. B lê TOTAL, que ainda vale 3: registrador de B = 3.
4. B calcula: $3 \times 2 = 6$.
5. B grava: TOTAL = 6.
6. A grava o valor que tinha no registrador: TOTAL = 4.

O resultado é 4 porque A leu o valor antigo e gravou por último, por cima do que B tinha gravado. A operação de B (multiplicar por 2) foi perdida, como se nunca tivesse acontecido.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a versão sem sincronização vai ser a mais rápida, porque não tem nada controlando o acesso, mas também vai ser a incorreta. Entre as outras duas, acho que o AtomicInteger vai ser mais rápido que o synchronized, porque a operação é feita direto no processador, sem travar e destravar.

Saída do programa

1ª execução:

esperado: 2000000

sem sincronizacao: 1925786 em 4 ms

synchronized: 2000000 em 87 ms

AtomicInteger: 2000000 em 73 ms

2ª execução:

esperado: 2000000

sem sincronizacao: 1509008 em 2 ms

synchronized: 2000000 em 179 ms

AtomicInteger: 2000000 em 64 ms

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com synchronized e com AtomicInteger deram o valor correto (2.000.000) nas duas execuções. A versão sem sincronização deu 1.925.786 e 1.509.008, errada e diferente a cada vez. Em tempo, a versão sem cuidado levou 2 a 4 ms, enquanto o synchronized levou 87 e 179 ms e o AtomicInteger 73 e 64 ms, ou seja, de 20 a 80 vezes mais lentas. A versão correta é mais lenta porque precisa garantir que só uma thread por vez mexa no contador: no synchronized a thread tem que pegar a trava e as outras ficam esperando, e no AtomicInteger o processador faz ler-somar-gravar como um passo só e as threads disputam a mesma posição de memória. Sem sincronização as threads não esperam nada, por isso são rápidas, mas erram.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve. O AtomicInteger garante que cada operação em um saldo seja atômica, mas a transferência são duas operações: tirar de uma conta e colocar na outra. Entre uma e outra, outra thread pode ler os saldos e ver o dinheiro "sumido" (já saiu de uma conta e ainda não entrou na outra), ou pode haver duas transferências tirando da mesma conta ao mesmo tempo e deixando o saldo negativo, porque a verificação do saldo e o débito também são passos separados. Seria preciso tornar a transferência inteira uma seção crítica, por exemplo com synchronized ou uma trava que proteja as duas contas juntas (pegando as travas sempre na mesma ordem para não dar impasse), ou usar uma transação, como num banco de dados.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acho que não vão aparecer. A pega X e dorme 100 ms, enquanto isso B pega Y. Depois A tenta pegar Y (que está com B) e B tenta pegar X (que está com A). Uma fica esperando a outra para sempre, então nenhuma imprime "terminou" e o programa deve mostrar que há impasse.

Saídas (antes e depois da correção)

Antes da correção (A pega X e depois Y; B pega Y e depois X):
IMPASSE: 2 threads presas

Depois da correção (B também pega X e depois Y):
A terminou
B terminou
sem impasse

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A pegou o recurso X e, depois da pausa, ficou esperando o Y. Ao mesmo tempo, a thread B pegou o Y e ficou esperando o X. Cada uma está segurando exatamente o recurso que a outra precisa, e nenhuma solta o seu antes de conseguir o outro. Como as duas esperam para sempre, nenhuma chega a imprimir "terminou" e a JVM detectou 2 threads presas. A pausa de 100 ms só garante que as duas peguem o primeiro recurso antes de tentar o segundo.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Porque, com a mesma ordem, quem pegar o X primeiro é quem vai conseguir o Y. A outra thread fica esperando já no X, sem segurar nada, então não existe mais a espera circular (A espera B e B espera A). Quando a primeira termina e solta X e Y, a segunda continua. Na saída corrigida apareceram "A terminou", "B terminou" e "sem impasse". Exemplo real: um sistema bancário que faz duas transferências ao mesmo tempo, uma da conta 1 para a 2 e outra da 2 para a 1, travando primeiro a conta de origem. Uma trava a conta 1 e espera a 2, a outra trava a 2 e espera a 1. A solução é travar sempre a conta de menor número primeiro.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, cada um faz 2 tarefas em paralelo com os outros, então acho que leva uns 200 ms. Com 1 trabalhador ele faz as 6 em sequência: 600 ms. Com 6 trabalhadores cada um faz uma tarefa e todas rodam juntas: uns 100 ms.

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
3 trabalhadores:
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 2 fez a tarefa 4
trabalhador 3 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
1 trabalhador:
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
6 trabalhadores:
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 5 fez a tarefa 6
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as 6 tarefas são feitas uma depois da outra: $6 \times 100 \text{ ms} = 600 \text{ ms}$. Com 3, cada trabalhador pega 2 tarefas e os três trabalham ao mesmo tempo: $2 \times 100 \text{ ms} = 200 \text{ ms}$. Com 6, cada um pega uma tarefa e todas acontecem juntas: 100 ms. O tempo cai porque o trabalho é dividido entre goroutines que executam em paralelo. O programa não precisa de synchronized nem de trava porque os trabalhadores não compartilham nenhuma variável: cada tarefa chega como uma mensagem pelo canal, e o próprio canal garante que cada mensagem seja entregue a um trabalhador só. Ninguém altera um dado que outro também está alterando, então não há condição de corrida.

Saídas (antes e depois da troca)

```
Sem buffer, make(chan int):
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
main.main()
    /box/main.go:7 +0x38
```

```
exit status 2
```

```
Com buffer de 1, make(chan int, 1):
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No canal sem espaço, quem envia fica bloqueado até alguém receber. O main faz `ch <- 1` e fica esperando, mas quem ia receber (o `<-ch` da linha de baixo) é o próprio main, que está parado. Não existe nenhuma outra goroutine para receber, então o Go detecta que todas estão dormindo e encerra com "deadlock!". É um impasse com uma tarefa só esperando por ela mesma. Com `make(chan int, 1)`, o canal tem espaço para guardar 1 mensagem: o envio coloca o 1 no buffer e não precisa esperar ninguém, o main continua, recebe o valor na linha seguinte e imprime 1.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acho que vai imprimir 1, porque a thread soma 1 no contador e o main espera ela terminar com `join` antes de imprimir. Talvez o Rust reclame de alguma coisa por causa do `mut`, mas acho que roda.

Saídas (o erro e o resultado com `move`)

Sem `move` (erro de compilação):

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
  → HelloWorld.rs:5:27
```

```
|
5 |     let t = thread::spawn(|| {
|                               ^^ may outlive borrowed value `contador`
6 |         contador += 1;
|         ----- `contador` is borrowed here
```

```
...
help: to force the closure to take ownership of `contador` (and any other referenced
variables), use the `move` keyword
```

```
...
error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
error: aborting due to 2 previous errors
```

Com `move` (compila, com avisos):

```
warning: value captured by `contador` is never read
  = help: did you mean to capture by reference instead?
warning: value assigned to `contador` is never read
warning: 2 warnings emitted
0
```

Saída do programa

```
contador = 400000
```

6.1 Com `move`, o programa compila mas imprime 0. Por quê? (Dica: `move` faz a thread levar uma cópia do número.)

Porque o contador é um `i32`, que é copiável. Com `move`, a closure da thread leva uma cópia própria do número, e não a variável do main. A thread soma 1 na cópia dela, que vira 1 e é descartada quando a thread termina. O contador do main nunca foi alterado e continua 0. O próprio compilador avisa isso: "value captured by contador is

never read" e pergunta se a intenção era capturar por referência. Para a thread alterar o mesmo valor que o main lê, ele precisa ser compartilhado de verdade, com Arc e Mutex como na 6b, que deu 400.000 certinho (4 threads × 100.000).

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java (Prática 1) o erro só apareceu na execução: o código compilou sem problema e o resultado veio errado, diferente a cada vez, e com $n = 1000$ nem apareceu. Em Rust o erro foi descoberto na compilação: o compilador nem deixou gerar o programa enquanto a variável era alterada por outra thread sem proteção. A vantagem é que a condição de corrida não chega aos testes nem à produção, porque o programa com esse erro simplesmente não compila. O custo é que quem programa precisa entender as regras de posse e empréstimo (ownership, borrow, move) e escrever mais código (Arc, Mutex, clone, lock) para coisas que em Java parecem simples, e no começo o compilador recusa muita coisa.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Acho que sai A, D, B, C. A e D são console.log normais e saem primeiro. Depois o setTimeout com 0 ms (B) e por último a promessa (C).

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms

(executei duas vezes e a saída foi igual)
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

As letras saíram na ordem A, D, C, B (eu tinha previsto A, D, B, C, então errei a ordem de B e C). A e D saem primeiro porque são código síncrono, executado direto pela única thread. O setTimeout e o then da promessa só vão para filas e esperam o código atual terminar. B saiu depois de C porque as promessas ficam na fila de microtarefas, que é esvaziada logo que o código atual acaba, antes de o laço de eventos pegar a próxima tarefa da fila de timers. Então o tempo 0 do setTimeout não significa "agora", e sim "assim que a thread estiver livre e for a vez da fila de timers".

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque o timer de 10 ms foi agendado e, logo em seguida, a função ocupado(300) prendeu a única thread em um laço por 300 ms. O timer venceu aos 10 ms, mas o callback dele só pode rodar quando a thread fica livre, então ele rodou depois de 300 ms. Num servidor Node.js, todos os usuários são atendidos por essa mesma thread. Se uma requisição fizer um cálculo pesado de, por exemplo, 2 segundos, todas as outras requisições ficam paradas esperando esse tempo, e o servidor parece travado. Cálculos pesados precisam ser divididos em partes, ir para worker threads ou para outro serviço.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.**Qual problema desta atividade pode acontecer? Como você o evitaria?**

Pode acontecer uma condição de corrida, igual à Prática 1 e à Figura 13.1. As duas requisições leem "1 lugar disponível" ao mesmo tempo, as duas acham que podem comprar, e as duas gravam a venda. O mesmo lugar é vendido duas vezes, e o estoque pode até ficar em -1. Para evitar, a parte de verificar o lugar e registrar a venda tem que ser uma seção crítica, feita de forma atômica: por exemplo, uma transação no banco de dados com trava na linha do lugar (SELECT ... FOR UPDATE), ou um UPDATE condicional do tipo "UPDATE lugares SET vendido = true WHERE id = X AND vendido = false" e conferir se alguma linha foi alterada. Assim só uma das compras consegue o lugar e a outra recebe a mensagem de esgotado.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que o mesmo programa em Java deu um resultado diferente a cada execução e só errava com n grande, ou seja, um bug que pode passar nos testes e só aparecer em produção.